DocMind — Multi-Tenant RAG Chat Platform

Technical Blueprint: Architecture, Differentiation & Zero-Cost Deployment Plan

Soufian

August 2026

Contents

1 Executive Summary **3**

2 What Makes This Unique **3**
2.1 Claim-Level Faithfulness Scoring 3
2.2 Cross-Document Conflict Detection 4
2.3 Cross-Lingual Retrieval (Arabic / French / English) 4

3 Product Overview & User Journey **5**

4 System Architecture **6**
4.1 Logical Components 6
4.2 Request Flow (a single question, end to end) 7
4.3 Data Isolation 7

5 Database Schema **8**

6 RAG Pipeline Design **9**
6.1 Ingestion 9
6.2 Hybrid Retrieval 9
6.3 Reranking 9
6.4 Generation and Verification 9

7 API Design **10**

8 Technology Stack **11**

9 Deployment Architecture **13**

10 Compute & Cost Analysis **14**

11 Security & Multi-Tenancy **15**

12 Observability & the Feedback Loop **15**

13 Development Roadmap **16**

14 Risks & Mitigations **16**

15 Closing Summary **17**

1 Executive Summary

DocMind is a self-hosted, multi-user RAG (Retrieval-Augmented Generation) chat platform. Users create an account, start a chat, upload one or more documents into that chat, and ask questions answered strictly from the uploaded material. It is designed as a portfolio-grade, production-shaped system rather than a single-script demo: real authentication, per-user data isolation, persistent chat history, document management, and observability are all first-class parts of the design, not afterthoughts.

Two constraints shape every decision in this blueprint:

1. **Zero cost, development through production.** Every component — embeddings, reranking, the LLM itself, the database, the vector store, the hosting, and the monitoring stack — runs on free tiers or self-hosted open-source software. No credit card is required at any layer.
2. **No dependency on GPU or heavy compute.** All models are chosen to run comfortably on a 4-core ARM CPU with 24 GB RAM (the ceiling of the Oracle Cloud Always Free tier), or to run for free on someone else's hardware via API (Groq's LPUs, Google's TPUs) rather than yours.

What differentiates DocMind from the thousand "chat with your PDF" tutorials on GitHub is described in detail in Section 2, but in short: most RAG demos silently blend retrieved chunks into one confident-sounding answer. DocMind instead treats retrieval as something that can fail, disagree with itself, or only partially support a claim — and it says so. This is closer to how RAG faithfulness is discussed in current research than to how it is usually shipped in tutorials, which makes it a stronger signal for both an ML engineering portfolio and a research-oriented graduate application.

2 What Makes This Unique

A generic RAG chatbot is: upload → chunk → embed → retrieve → stuff into a prompt → generate. That pipeline is necessary but no longer differentiating on its own — it is table stakes. DocMind keeps that pipeline as its foundation and adds three specific capabilities on top of it, each chosen because it is (a) genuinely useful, (b) cheap to compute, and (c) rarely implemented end-to-end in public tutorials or portfolio projects.

2.1 Claim-Level Faithfulness Scoring

Most RAG systems cite at the document level: "this answer is based on document X." That is a weak guarantee — a paragraph-long answer might have one well-supported sentence and three fabricated ones, and document-level citation cannot tell the user which is which.

DocMind decomposes every generated answer into individual claims (short, atomic statements), and checks each claim against the retrieved chunks using a lightweight natural language inference (NLI) model that classifies the relationship as entailed, contradicted, or unsupported. Each claim in the final answer is rendered with a small inline indicator — supported, weakly supported, or unverified — instead of a single

blanket "sources" footer. This turns citation from a formality into an actual trust signal, and it is the single feature most likely to make an evaluator (professor, hiring manager, or grad admissions committee) stop and look closer, because it directly engages with the hallucination problem that is central to current RAG research rather than assuming it away.

2.2 Cross-Document Conflict Detection

When a chat has multiple documents — which is the normal case, not the exception, once someone uploads more than one source — those documents sometimes disagree. A generic RAG pipeline retrieves the top-k most similar chunks regardless of whether they agree, and the LLM quietly picks one version or blends them into a muddled answer.

DocMind runs the same NLI check used for claim-level scoring in a second mode: comparing retrieved chunks against *each other*, not just against the generated answer. When two retrieved chunks from different documents contradict each other on a point relevant to the question, the system flags this explicitly in the response ("Document A states X; Document B states Y — these conflict") instead of silently resolving it. This is directly useful for the kind of documents people actually upload in bulk — contract versions, policy revisions, comparative research papers, multiple vendor proposals — and it is a feature that is almost never present in open-source RAG templates, because most of them assume a single, internally consistent knowledge base.

2.3 Cross-Lingual Retrieval (Arabic / French / English)

Nearly all public RAG tooling, embedding benchmarks, and tutorials are built and evaluated English-first. DocMind uses a multilingual embedding model so that a user can ask a question in French, retrieve the correct chunk from an Arabic or English document, and get an answer in the language they asked in — without needing separate pipelines per language or manual translation steps. This is a genuine, underserved gap (most RAG stacks bolt on translation as an afterthought, if at all) and it is also the differentiator most grounded in your own background: trilingual fluency plus prior work on multilingual BERT sentiment analysis during your PFE at Yool Education. It is the single feature in this blueprint that another applicant is least likely to be able to replicate credibly.

Together, these three features turn DocMind from "another RAG chatbot" into a system whose central design question is *how much should you trust this answer, and why* — which is a more interesting and more defensible thesis for a portfolio piece than raw feature count.

3 Product Overview & User Journey

DocMind is structured around three nested objects: a **user account**, one or more **chats** belonging to that user, and one or more **documents** attached to each chat. Conversation happens inside a chat and is always grounded in the documents attached to that specific chat — a user can run several independent chats (e.g., one per project, one per course, one per client) without their document sets or chat histories bleeding into each other.

The end-to-end journey:

1. **Sign up / log in.** Email + password (hashed, never stored in plaintext), or an optional passwordless magic-link flow for a nicer demo experience. A session token (JWT) is issued and used for all subsequent requests.
2. **Create a chat.** The user names a new chat (e.g., "Thesis literature review" or "Q3 vendor contracts"). This creates an empty conversation container tied to their account.
3. **Upload documents into that chat.** PDF, DOCX, or TXT files are uploaded, parsed, chunked, embedded, and stored — scoped to that chat, invisible to any other user or any other chat the same user owns. The user sees per-document status (processing, ready, failed) so upload is not a black box.
4. **Ask a question.** The user types a question in the chat. DocMind retrieves the most relevant chunks from that chat's documents only, runs the retrieval and verification pipeline described in Section 2, and returns an answer with per-claim confidence indicators and, where relevant, a conflict warning.
5. **Continue the conversation.** Follow-up questions reuse chat history for context (so "what about the second one?" resolves correctly), and the user can add more documents to the same chat at any point — the retrieval index simply grows.
6. **Review and manage.** The user can revisit any past chat, see which documents were used to answer which message, delete a document (which removes its chunks from retrieval immediately), or delete an entire chat.

This structure is deliberately close to how real products in this space are built (a scoped "workspace per conversation" model), which makes DocMind a credible answer to "have you built something production-shaped," not just "have you called an LLM API."

4 System Architecture

DocMind is a monolithic Flask application backed by PostgreSQL, deliberately avoiding a microservices split. At portfolio scale (and at Oracle Free Tier resource limits) a single well-organized service is easier to reason about, easier to deploy, and easier to explain in an interview than a fleet of containers talking over a message bus — complexity should be earned, not assumed.

4.1 Logical Components

Component	Responsibility
Auth service (Flask blueprint)	Registration, login, JWT issuance/refresh, password hashing (bcrypt)
Chat service (Flask blueprint)	Create/list/delete chats, store message history, orchestrate the RAG pipeline per query
Ingestion service (Flask blueprint + Celery worker)	Accepts uploads, extracts text, chunks, embeds, writes vectors to pgvector, tracks per-document status
Retrieval engine	Hybrid search (BM25 lexical + dense vector) scoped to a chat's document set, followed by cross-encoder reranking
Verification engine	NLI-based claim scoring and cross-document conflict detection (Section 2 features)
Generation gateway	Routes generation requests across free LLM providers with automatic fallback (Section 8)
Object storage	Original uploaded files, kept for re-processing and audit, stored on local disk or a free-tier S3-compatible bucket
Cache layer (Redis)	Caches embeddings for repeated content, caches LLM responses for identical queries, and enforces per-user rate limits to protect free-tier API quotas
Monitoring stack	Prometheus scrapes app + Celery + Postgres metrics; Grafana dashboards for latency, retrieval quality signals, and provider fallback rate
Experiment tracking	MLflow logs retrieval configuration experiments (chunk size, hybrid weighting, reranker on/off) during development

4.2 Request Flow (a single question, end to end)

A user's question moves through six stages: the query is embedded and expanded; hybrid retrieval pulls candidate chunks from that chat's documents only, scored by a combination of lexical and semantic similarity; a cross-encoder reranks the top candidates for precision; the reranked chunks and chat history are assembled into a prompt and sent to the generation gateway; the raw answer is decomposed into claims and each claim is checked against the retrieved evidence, while the retrieved chunks are also checked against each other for conflicts; and finally the answer, its per-claim confidence markers, any conflict warnings, and the source citations are returned to the user and persisted as a chat message. Each stage writes a timing metric to Prometheus, so slow stages are visible rather than guessed at.

4.3 Data Isolation

Every table that stores user content (documents, chunks, messages) carries both a `user_id` and a `chat_id` foreign key, and every query in the retrieval layer filters on both — a document uploaded to one chat is structurally invisible to retrieval in any other chat, even for the same user, and completely invisible across users. This is enforced at the query layer rather than relying on the LLM to "only use the provided context," which is a common and fragile pattern in weaker RAG implementations.

5 Database Schema

PostgreSQL with the `pgvector` extension serves as both the relational store and the vector store — one database, no separate vector database service to run, back up, or pay for.

Table	Key columns	Purpose
`users`	id, email, password_hash, created_at	Account records
`chats`	id, user_id, title, created_at, updated_at	One conversation container per project/topic
`documents`	id, chat_id, filename, storage_path, status, language, page_count, uploaded_at	Uploaded file metadata and processing status
`chunks`	id, document_id, chat_id, content, embedding (vector), token_count, page_number	Chunked text with its embedding; `chat_id` is denormalized onto this table specifically to make retrieval-scoping a single indexed filter rather than a join
`messages`	id, chat_id, role (user/assistant), content, created_at	Full conversation history
`citations`	id, message_id, chunk_id, claim_text, verdict (supported/weak/unverified), confidence	Per-claim verification results linking an assistant message to the evidence used to check it
`conflicts`	id, message_id, chunk_id_a, chunk_id_b, description	Logged cross-document contradictions surfaced in a given answer
`feedback`	id, message_id, user_id, rating, note, created_at	Thumbs up/down signal, used to drive the retrieval feedback loop described in Section 9

Indexing notes: an IVFFlat or HNSW index on `chunks.embedding` (pgvector supports both) keeps vector search fast even as chunk count grows into the hundreds of thousands; a standard B-tree index on `(chat_id, document_id)` keeps scoped filtering cheap; a GIN index on `chunks.content` enables the lexical half of hybrid search via Postgres full-text search, avoiding the need for a separate Elasticsearch/OpenSearch deployment.

6 RAG Pipeline Design

6.1 Ingestion

Uploaded PDFs, DOCX, and TXT files are parsed with open-source libraries (`pypdf`/`pdfplumber` for PDF, `python-docx` for Word) rather than a paid document-parsing API. Text is split using a recursive character/token-aware chunker with overlap (roughly 400–600 tokens per chunk, ~15% overlap), which balances retrieval precision against context waste — small enough that a chunk is topically coherent, large enough that it carries the surrounding context a claim needs to be checked against. Detected document language is stored per document, which the cross-lingual retriever (Section 2) uses to normalize query-document language mismatches.

Ingestion runs as a Celery background task rather than inline in the upload request, so a large PDF does not block the HTTP response — the user sees a "processing" status that flips to "ready" once chunking and embedding complete, which is both better UX and a more honest representation of how ingestion actually behaves at scale.

6.2 Hybrid Retrieval

Pure dense retrieval misses exact keyword matches (names, IDs, specific figures); pure lexical (BM25) retrieval misses paraphrase and cross-lingual matches. DocMind combines both: dense similarity search via pgvector and lexical search via Postgres full-text search, merged with Reciprocal Rank Fusion (RRF), scoped at the SQL level to `chat_id`. This hybrid step alone typically improves retrieval recall over dense-only search by a meaningful margin on heterogeneous document sets, and it costs nothing extra to run since both halves are native Postgres operations.

6.3 Reranking

The top ~20 candidates from hybrid retrieval are rescored by a cross-encoder reranker, which reads the query and each candidate chunk together (rather than comparing precomputed embeddings) and produces a much more precise relevance score. This step is CPU-feasible because reranking only 20 short chunks per query is a small, one-shot batch — nothing close to the compute cost of training or fine-tuning.

6.4 Generation and Verification

The top ~5 reranked chunks, plus recent chat history, are assembled into a prompt sent to the generation gateway (Section 8). The raw response is then passed through the verification engine described in Section 2: claim decomposition, per-claim NLI scoring against the cited chunks, and a separate pairwise NLI pass across the retrieved chunks themselves to catch source-level conflicts. Both verification passes use a small, CPU-friendly NLI model and add on the order of a few hundred milliseconds to total response time — a worthwhile trade for the trust signal it produces.

7 API Design

A REST API under a single Flask application, versioned under `/api/v1`, organized by resource:

Endpoint	Method	Purpose
`/api/v1/auth/register`	POST	Create account
`/api/v1/auth/login`	POST	Authenticate, issue JWT
`/api/v1/auth/refresh`	POST	Refresh an expiring token
`/api/v1/chats`	GET, POST	List user's chats / create a new chat
`/api/v1/chats/{chat_id}`	GET, DELETE	Fetch chat detail and history / delete a chat
`/api/v1/chats/{chat_id}/documents`	GET, POST	List documents in a chat / upload a new document
`/api/v1/documents/{document_id}`	GET, DELETE	Check processing status / remove a document and its chunks
`/api/v1/chats/{chat_id}/messages`	GET, POST	Fetch message history / ask a new question
`/api/v1/messages/{message_id}/citations`	GET	Retrieve per-claim verification detail for a given answer
`/api/v1/messages/{message_id}/feedback`	POST	Submit thumbs up/down and an optional note

Every endpoint under `/chats` and below requires a valid JWT and is scoped server-side to `user_id`, so there is no client-trusted parameter that could leak another user's data — ownership is re-checked on every request against the database, not inferred from the token alone.

8 Technology Stack

Every layer below is free with no credit card required, and every model choice is CPU-viable. Where an LLM API is used, it is a hosted free tier rather than something you run yourself — this is the key trick to getting frontier-adjacent generation quality without owning a GPU.

Layer	Choice	Why it fits
Backend framework	Flask	Matches your existing stack; lightweight, explicit, easy to reason about in interviews
Database	PostgreSQL + `pgvector`	One database for relational data and vectors; avoids running a separate vector DB service
Embeddings	`BAAI/bge-m3` (multilingual, via `sentence-transformers`)	Open-weight, strong multilingual performance (needed for the cross-lingual feature), runs on CPU in well under a second per chunk
Reranker	`BAAI/bge-reranker-base` (cross-encoder)	CPU-viable for small batches; large precision boost over dense-only retrieval
NLI / verification model	A compact NLI model such as `MoritzLaurer/mDeBERTa-v3-base-mnli-xnli` (multilingual)	CPU-feasible, and multilingual so claim verification works across Arabic/French/English documents, not just English ones
Primary LLM	Google Gemini Flash (free tier via Google AI Studio)	Frontier-class quality at no cost, generous daily quota, 1M-token context useful for long documents
Fallback LLM	Groq (free tier, no credit card, open-weight models such as Llama and Qwen)	Absorbs traffic when the primary provider is rate-limited, keeping the app responsive
Background jobs	Celery + Redis	Async document ingestion; matches your ModelForge experience

Layer	Choice	Why it fits
Caching / rate limiting	Redis	Caches repeated queries and embeddings; throttles usage to stay inside free daily quotas
Containerization	Docker Compose	Same pattern as your prior projects; single-command local and production spin-up
Hosting	Oracle Cloud Always Free (ARM Ampere, 4 OCPU / 24 GB RAM tier)	Genuinely free indefinitely, enough headroom to run Postgres, Flask, Celery, Redis, and the embedding/reranking/NLI models together
Experiment tracking	MLflow (self-hosted)	Tracks retrieval configuration experiments during development
Monitoring	Prometheus + Grafana	Latency, provider fallback rate, retrieval quality signals in production
Object storage	Local disk volume (or a free-tier S3-compatible bucket such as Cloudflare R2's free allowance)	Stores original uploaded files for re-processing

A note on provider risk: free LLM tiers change quotas and model catalogs without much warning. The generation gateway is built provider-agnostic from day one — a thin abstraction that can route to Gemini, Groq, or a third free provider behind the same interface — specifically so a quota change on one provider is a configuration change, not a rewrite.

9 Deployment Architecture

The entire stack runs as a single Docker Compose file on one Oracle Cloud Always Free ARM instance:

Service	Container	Approx. memory footprint
Flask app (Gunicorn)	`docmind-api`	~300 MB
Celery worker	`docmind-worker`	~500 MB (includes loaded embedding/reranker/NLI models)
PostgreSQL + pgvector	`docmind-db`	~400 MB at portfolio scale
Redis	`docmind-cache`	~50 MB
Prometheus	`docmind-metrics`	~150 MB
Grafana	`docmind-dashboard`	~150 MB

That totals well under the 24 GB available on the free ARM tier, even accounting for the OS and Docker overhead, leaving comfortable headroom — this is a deliberately conservative design rather than one that runs at the edge of its resource budget.

A reverse proxy (Caddy or Nginx) in front of the Flask app handles TLS termination using a free Let's Encrypt certificate, and a free-tier domain or a subdomain of an existing domain points at the instance's public IP. GitHub Actions (free for public repositories, and free within a generous minutes allowance for private ones) runs the CI/CD pipeline: lint and test on every push, build the Docker image, and deploy to the Oracle instance over SSH on merge to main — the same CI/CD shape used in your Time Series Forecasting Platform.

10 Compute & Cost Analysis

The design goal is not just “free today” but “free at the traffic level a portfolio project actually receives,” which matters because free tiers are rate-limited, not unlimited. Three mechanisms keep DocMind inside those limits without degrading the demo experience:

Caching. Redis caches the embedding of any previously-seen chunk and the full response to any previously-seen question-plus-document-set combination, so repeated demo queries (which are common when showing the project to reviewers) cost zero additional API calls after the first run.

Provider fallback. The generation gateway automatically retries against a second free provider when the first returns a rate-limit error, rather than surfacing a failure to the user. This is invisible in normal use and only becomes relevant under bursty traffic (e.g., a spike of interview demos in a short window).

Local models for the expensive-per-call steps. Embeddings, reranking, and NLI verification all run locally on the ARM instance rather than through a metered API. These are the steps that would otherwise be called once per chunk and once per claim — i.e., the highest call volume in the whole pipeline — so keeping them local is what makes the free-tier LLM budget last, since only the final generation step (roughly one call per user question) touches a rate-limited API at all.

The net effect: a portfolio-level deployment (a handful of active users, dozens of documents, a few hundred questions a month) sits comfortably inside every free tier involved, with no realistic path to an unexpected bill, because there is no payment method attached to any of the services in the first place.

11 Security & Multi-Tenancy

Passwords are hashed with bcrypt and never logged or stored in plaintext. Authentication uses short-lived JWTs with a refresh flow rather than long-lived tokens. Every content table is scoped by `user_id` and `chat_id`, and every retrieval query filters on both at the SQL level rather than trusting a client-supplied identifier — this is what actually prevents one user's uploaded contract from ever being retrievable in another user's chat, rather than relying on the LLM prompt to "only discuss the provided context," which is not a security boundary. Uploaded file types are validated by content, not just extension, before parsing, and file size limits are enforced both client-side and server-side to protect the ingestion queue from abuse. Rate limiting at the Redis layer applies per-user, so one account's usage pattern cannot exhaust the shared free-tier LLM quota for every other user of the deployment.

12 Observability & the Feedback Loop

Beyond standard latency and error-rate dashboards, DocMind logs retrieval-specific signals that most portfolio RAG projects never capture: which chunks were retrieved but not cited in the final answer (a proxy for retrieval noise), how often the conflict detector fires, and how often users leave negative feedback on an answer that had a low aggregate claim-confidence score. That last correlation is the most valuable one — if low-confidence answers reliably predict negative feedback, it validates that the claim-verification feature is measuring something real rather than just adding UI decoration, and MLflow-tracked experiments (chunk size, hybrid retrieval weighting, reranker on/off) can be evaluated against actual user feedback rather than only offline retrieval metrics.

13 Development Roadmap

Phase	Scope
Phase 1 — Core RAG loop	Auth, chats, document upload, ingestion pipeline, hybrid retrieval, basic generation. A working single-document chatbot.
Phase 2 — Differentiators	Claim-level verification, cross-document conflict detection, multilingual embedding upgrade. This is where DocMind stops being generic.
Phase 3 — Production hardening	Rate limiting, provider fallback gateway, Prometheus/Grafana dashboards, Docker Compose + CI/CD deployment to Oracle Free Tier.
Phase 4 — Feedback loop	Thumbs up/down capture, retrieval-quality dashboards, MLflow-tracked experiments comparing chunking/retrieval configurations against real feedback data.
Phase 5 (stretch) — Polish for portfolio	Clean frontend (chat UI with per-claim confidence highlighting and conflict banners), a short demo video, and a condensed write-up describing the faithfulness-scoring design decision for use in graduate applications or interviews.

14 Risks & Mitigations

Risk	Mitigation
Free LLM provider changes quota or retires a model	Provider-agnostic generation gateway; fallback provider configured from day one
ARM instance runs out of memory under multiple simultaneous large-document ingestions	Celery task concurrency capped; ingestion queue processes one large document at a time per worker
NLI-based verification adds latency users notice	Verification runs after the answer is generated and streamed to the user; confidence indicators can populate a moment after the text itself, rather than blocking the whole response

Risk	Mitigation
Cross-lingual retrieval underperforms on dialectal Arabic (vs. Modern Standard Arabic)	Documented as a known limitation in the README rather than overclaimed; framed as a natural “future work” extension, which is itself a credible thing to say in a graduate application

15 Closing Summary

DocMind is scoped to be buildable end-to-end with tools you already have hands-on experience with — Flask, PostgreSQL, Docker Compose, Celery, Redis, MLflow, Oracle Free Tier — while adding a genuinely novel angle (claim-level faithfulness and cross-document conflict detection) that most public RAG projects skip entirely, plus a personal-background angle (trilingual retrieval) that is difficult for another applicant to replicate credibly. The zero-cost stack is not a compromise made for a student budget; every choice in this blueprint is also a defensible production choice, which is the strongest possible framing for a portfolio piece aimed at either graduate admissions or an ML engineering role.